



3rd Exercise

Architecture and CAD for FPGAs

Context

Routing is a key problem to be solved when implementing a user design on an FPGA. For routing, the user design is usually given as a placed netlist providing primitives that exist on the FPGA (e.g., LUTs, DSP-blocks, Flops, etc.). Those primitives are usually annotated with location information and parameter settings. (something like `my_LUT, PLACED X2_Y3_ALUT, INIT=01010011`)

In addition, the netlist contains nets which define connections between the placed primitives. To be more precise: between pins of primitives. A net usually has one driver primitive pin and can have more or multiple destination primitive pins. The latter is forming a spanning tree (think of a reset signal that drives many flop reset inputs). The net is often somehow named by the symbolic netlist name of the driver primitive (as it should be unique).

The task of a router is to find multiplexer settings of the switch matrix multiplexers and allocated routing resources such that all nets are implemented. This must ensure that resources used for different nets do not collide with each other. Moreover, we usually want to optimize the speed of a user circuit, which depends on the longest path on the fabric, and consequently, on the routing. And, in general, we have to obey min-max latency constraints to meet setup and hold-time requirements (don't worry if that is new to you, we get to that).

It should be mentioned that instead of routing between primitive pins, any resource (including a routing wire or switch matrix multiplexer) may be a begin or end port of a path. For instance, this is useful for implementing partially reconfigurable systems.

Aufgabe 1 (Basic Routing Experiments)

This is the first exercise concerned with routing a user module on an FPGA. Further and related exercises will follow and you should write your code in a way that you can build up on it in consecutive exercises.

Routing is a very compute heavy process and we would usually implement a router in C++. For instance, a datacenter FPGA design may require the implementation of a few million nets while negotiating which net gets which physical FPGA routing resource. Anyway, for our lab, you can use C++ or Python as the understanding of an algorithm and the strategies for FPGA routing are more important than the actual language used for implementation.

For our first exercise on routing you should perform the following:

- a) Generate an FPGA fabric with LUTs and I/Os as in the FABulous tutorial. The fabric should have a size of 8x8 CLBs (which is our LUT4AB tile). At this point, we don't use other blocks, such as DSPs inside the fabric. The I/Os don't matter, as long as they exist.
- b) After you generated the fabric, examine the hidden `.FABulous` folder. This folder contains the model files used for nextpnr. Try to understand those files, in particular the `bel` and the `pips` files.

Write a program that can read in those files and build your own data structure for routing (e.g., a dictionary). Here, we don't use deduplication. This means, we can have a flat routing graph and don't have to consider how to share information (e.g., by using references and relative coordinates).

Foresee that your data structure may need some attributes in the future. This could include timing information (min and max delays), power (how much energy does a toggle of a resource takes), or things like if a resource is already in use, or for which net it is used.

- c) Implement a Breadth-First Search (BFS) for finding a path from the `ALUT.O` output pin to the `HLUT.I0` input pin in the i) top-left tile.

Instrument your code to record 1) the number lookups in your dictionary and 2) the size of your search front (e.g., the entries of your job queue for your BFS). Try to estimate the number of lookups and memory resources needed before running the experiment. List your findings and discuss your estimate in the report.

Report the total number of lookups and the peak number of items in your search front (or job queue). Repeat the experiment for other destinations by moving the destination of `HLUT.I0` to the ii) bottom-left, iii) top-right and bottom-right corner of the fabric.

- d) **optional:** Repeat c) by using Depth-first search (DFS). Compare BFS and DFS and report your findings. Think first what is the difference between BFS and DFS and how that impacts your implementation.

You will submit this project (sources and report) together with a consecutive exercise on routing.